

A Hierarchical Timed-Automata Model for SDN-Managed Resource Orchestration in 6G ISAC Networks

Artur Mustafin¹[0009–0008–8322–0864] and Vadim Tinishov¹[0009–0003–7319–1944]

ITMO University, Faculty of Software Engineering and Computer Systems, Saint Petersburg, Russia

armustafin@itmo.ru, vadimtdot@gmail.com

Abstract. Integrated sensing and communication (ISAC) turns the radio access network, transport segment, and edge infrastructure into a shared resource for data delivery, environment sensing, and computation. This paper assembles a verification-oriented model of such an infrastructure from four layer specifications: physical-layer sensing and signaling, MAC/resource scheduling, SDN/RAN intelligent control, and application-level service contracts. The model represents each layer as a finite timed automaton with explicit clocks, guards, bounded counters, synchronization channels, and assume–guarantee contracts. The physical layer abstracts continuous radio behavior into conservative quality classes; the MAC layer converts these classes into queueing and allocation decisions; the SDN/RIC layer performs policy selection, rule installation, recovery, and rollback; and the application layer defines service criticality, deadlines, and SLA outcomes. The resulting hierarchy is intended for UPPAAL implementation and for checking deadlock freedom, bounded response, queue safety, sensing degradation handling, rule-miss processing, and fault recovery. The manuscript includes screenshots of the currently available UPPAAL templates for the four layers and keeps the model interfaces ready for later regeneration from the XML repository.

Keywords: Integrated sensing and communication · 6G · SDN · RAN intelligent controller · timed automata · UPPAAL · resource orchestration · resilience

1 Introduction

Integrated sensing and communication changes the role of mobile infrastructure. A 6G base station, UAV node, RIS-assisted segment, or edge cluster is not only a communication endpoint; it can also participate in target detection, localization, environmental mapping, digital-twin updates, and safety monitoring. Consequently, the same infrastructure must satisfy data-rate and latency requirements together with sensing metrics such as detection probability, false-alarm probability, Cramer–Rao bounds, age of sensing information, angular and range resolution, and sensing coverage [11, 8, 1, 9].

This tension is cross-layer. A sensing-dominant mode may require additional pilots, different beams, a different waveform profile, or more edge processing, whereas a communication-dominant mode may reduce sensing overhead to preserve throughput and delay. At the same time, resilience mechanisms such as multipath delivery, selective standby failover, dependent recovery in multi-level cluster systems, and re-embedding influence the resources that remain available for ISAC operation [6, 10, 5]. A static split between sensing and communication resources is therefore insufficient: it cannot express service criticality, global controller policy, queue pressure, failure recovery, or rollback.

We use software-defined networking (SDN) and the RAN intelligent controller (RIC) as the programmable control layer for the whole ISAC system. The SDN/RIC plane collects telemetry from radio, MAC, transport, edge, cloud, and service components and selects resource partitions, flow rules, service admission decisions, routing, and recovery actions. Recent work on AI-assisted slicing and resource allocation shows the relevance of such control loops, but it does not by itself provide a compact model for formal verification of deadlines, rule-miss handling, recovery, and SLA impact [7, 12, 3].

The contribution of this paper is a unified, Springer LNCS-formatted article assembled from the PHY, MAC, SDN/RIC, and application-layer specifications. The article defines the layer hierarchy, the finite abstractions used at each layer, the shared interface, and the verification properties that should be encoded in UPPAAL. The model is not a replacement for link-level or packet-level simulation. Instead, it is a conservative timed-automata abstraction for early verification of orchestration logic in SDN-managed 6G ISAC networks.

2 Hierarchical Model

The complete system is represented as a synchronized product of layer automata, an abstract environment, and observer automata:

$$\mathcal{M} = \mathcal{A}_{PHY} \parallel \mathcal{A}_{MAC} \parallel \mathcal{A}_{SDN} \parallel \mathcal{A}_{APP} \parallel \mathcal{A}_{ENV} \parallel \mathcal{A}_{OBS}. \quad (1)$$

Each layer automaton is a tuple

$$\mathcal{A}_L = (L, l_0, C, V, \Sigma, E, Inv), \quad (2)$$

where L is the finite set of locations, l_0 is the initial location, C is the set of clocks, V is the set of finite-domain variables, Σ is the set of synchronization channels, E is the transition relation, and Inv assigns clock invariants to locations. The model deliberately uses finite classes instead of continuous-valued traces to keep reachability and bounded-response checks tractable in ordinary timed automata [2].

The hierarchy is motivated by state-space control. A monolithic automaton for radio behavior, scheduling, controller logic, and service contracts would obscure the source of violations and lead to unnecessary state explosion. The layered decomposition permits coarse abstractions for stable subsystems and

Table 1. Layer roles and finite abstractions in the unified model.

Layer	Main automata	Finite abstraction and output
PHY	$\mathcal{A}_{CH}, \mathcal{A}_{SIG}, \mathcal{A}_{BM}, \mathcal{A}_{SQ}, \mathcal{A}_{PH}$	Channel, signaling, beam, sensing-quality, and aggregate PHY states. Outputs quality classes, freshness, sensing capacity, CRB class, detection class, and PHY resource pressure.
MAC	$\mathcal{A}_{SCH}$, queue, buffer, $\mathcal{A}_{RSRC}$	Scheduler, queue, buffer, and resource states. Outputs allocation mode, queue class, drop reason, resource conflicts, and MAC-level sensing or communication pressure.
SDN/RIC	monitor, policy, rule, recovery, rollback	Global control loop for telemetry collection, policy evaluation, rule installation, failure handling, standby switch, reactive re-embedding, and rollback.
Application	$\mathcal{A}_{REQ}, \mathcal{A}_{SLA}$, admission, observers	Service requests, criticality, SLA deadlines, degradation tolerance, admission result, and final service impact.

more detailed templates for components that influence the target metric. This is aligned with hierarchical and reconfigurable timed-automata modeling, where a component can be refined only when a failure, policy change, or diagnostic scenario requires additional detail [4, 13].

Every layer is described by an assume-guarantee contract

$$\mathcal{C}_L = (Ass_L, Guar_L, I_L, O_L, D_L), \quad (3)$$

where Ass_L states admissible inputs, $Guar_L$ states guaranteed outputs, I_L and O_L are input and output channels, and D_L is the set of relevant deadlines. A composition is admissible when the guarantees of lower layers satisfy the assumptions of upper layers and the controller never hides a violation that changes service impact.

The shared interface has the form

$$I = (src, dst, ch, sync, payload, deadline, outcome). \quad (4)$$

The field ch identifies the UPPAAL synchronization channel, $payload$ is encoded by shared bounded variables before synchronization, $deadline$ determines the observer clock bound, and $outcome$ is the permitted response class. The main outcomes are ACCEPTED, DEGRADED, REJECTED, FAILED, INSTALLED, DROPPED, RECOVERED, and ROLLED_BACK.

3 Physical-Layer Automata

The PHY layer abstracts hybrid ISAC signaling without simulating I/Q samples. Its inputs are the signal profile chosen by the controller, the current channel and interference class, beam state, sensing freshness, and MAC resource availability. Its outputs are finite classes that can be consumed by MAC and SDN/RIC automata. This avoids mixing continuous radio equations with controller verification while preserving the monotonic effect of resource and channel degradation.

The channel automaton $\mathcal{A}_{CH}$ models normal, degraded, blocked, and recovered channel states. The signaling automaton $\mathcal{A}_{SIG}$ represents the selected ISAC mode: communication-dominant, balanced, sensing-dominant, or constrained. The beam-management automaton $\mathcal{A}_{BM}$ models beam acquisition, tracking, misalignment, and recovery. The sensing-quality automaton $\mathcal{A}_{SQ}$ classifies detection and freshness states. The aggregate automaton $\mathcal{A}_{PH}$ reports a single PHY state to the upper layers after combining channel, signaling, beam, and sensing evidence.

The finite discretization can be written as

$$\delta_{PHY}(\gamma, P_d, P_{fa}, CRB, AoS, B) = (q_{ch}, q_{det}, q_{crb}, q_{fresh}, q_{cap}), \quad (5)$$

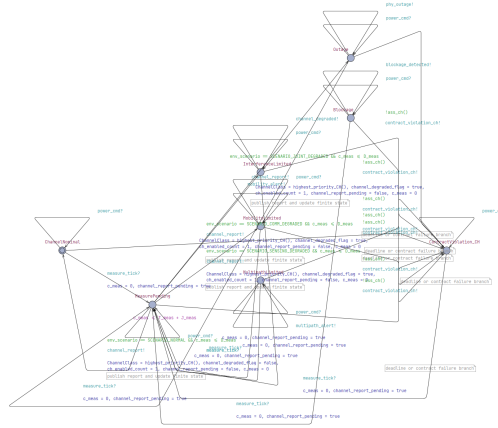
where γ is an SINR-like class input, P_d and P_{fa} are detection and false-alarm indicators, CRB is the estimation-accuracy class, AoS is the age of sensing information, and B is the available bandwidth or symbol budget. The result uses bounded enumerations such as $q_{ch} \in \{\text{GOOD}, \text{WEAK}, \text{OUTAGE}\}$ and $q_{fresh} \in \{\text{FRESH}, \text{STALE}\}$. Thresholds are selected conservatively: if a continuous region overlaps two classes, the less favorable class is used. This preserves safety properties for deadline and degradation checks.

The PHY contract assumes that the MAC layer provides a bounded resource decision within D_{mac} and that the environment changes channel class only through modeled events. It guarantees a PHY report within D_{phy} after a relevant channel, signal, or beam event. The report is sent over a channel such as `phy_kpi_report!` and includes *SensingState*, *FreshnessClass*, *ResourceClass*, and *ConflictReason* as shared variables. The current UPPAAL screenshot of the channel automaton is shown in Fig. 1.

4 MAC and Resource Scheduling

The MAC/resource-scheduling layer converts PHY quality classes into bounded allocation decisions. It decides whether the system remains in a normal balanced mode, increases sensing resources, prioritizes communication, enters a constrained mode, or rejects a request. It also exposes queueing and buffer states so that SDN/RIC and service observers can detect whether a formal violation is caused by radio quality, scheduler overload, or controller delay.

The scheduler automaton $\mathcal{A}_{SCH}$ contains locations for idle operation, request classification, resource calculation, allocation, and timeout. Queue and buffer automata count pending packets or sensing jobs up to finite limits. The resource



automaton $\mathcal{A}_{RSRC}$ tracks whether pilot, payload, beam, and edge-processing budgets are sufficient for the current service class. A typical policy guard is

$$g_{boost} = (SensingState \in \{WEAK, STALE\}) \wedge (R_{free} \geq R_{boost}) \wedge (Q_{comm} \leq Q_{safe}). \quad (6)$$

which permits a sensing boost only when the communication queue remains safe. If g_{boost} is false and the service is critical, the MAC layer reports constrained operation rather than silently dropping sensing performance.

The MAC contract assumes that PHY reports are received with bounded age and that the SDN/RIC layer acknowledges policy changes within D_{ctrl_ack} . It guarantees that each admissible service or packet request reaches one of the following outcomes within D_{mac} : allocated, delayed within a bounded queue, degraded with reason, or dropped with reason. The layer reports *AllocMode*, *QueueClass*, *DropReason*, and *ResourceConflict* via `mac_report!`. The scheduler template is shown in Fig. 2.

5 SDN/RIC Control Plane

The SDN/RIC layer is the central control automaton. It collects PHY and MAC reports, receives service requests and data-plane rule misses, evaluates policies,

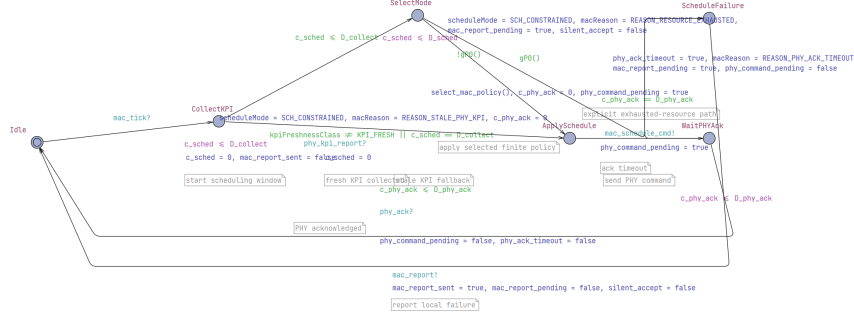


Fig. 2. UPPAAL template for the MAC scheduler automaton $\mathcal{A}_{SCH}$, including request classification, resource allocation, constrained operation, and scheduling outcomes.

installs flow rules, issues resource commands, and handles failures. Its locations include stable configuration, telemetry collection, policy evaluation, rule miss, rule-install pending, acknowledgement wait, failure detected, standby switch, reactive re-embedding, rollback, and rejected-by-policy.

The controller uses a multi-objective policy score

$$J = w_c U_{comm} + w_s U_{sense} - w_e E - w_r Risk - w_v Viol, \quad (7)$$

where U_{comm} and U_{sense} are communication and sensing utility classes, E is an energy or resource-cost class, $Risk$ is an infrastructure-risk class that can include dependent recovery effects in multi-level edge or cloud clusters [5], and $Viol$ penalizes deadline and SLA violations. The expression is not optimized numerically inside the timed automaton. Instead, it is discretized into policy classes such as NORMAL, SENSING_BOOST, CONSTRAINED, RECOVERY, and REJECT.

Four procedures are mandatory in the SDN/RIC interface. A rule miss carries flow, slice, service, and rule classes and must end in rule installation and forwarding or in a drop report. A recovery procedure carries fault, affected-slice, affected-service, and recovery classes and must end in a stable configuration, rollback, or failure report. A sensing-degradation procedure receives PHY/MAC evidence and must select sensing boost, constrained mode, or rejection. A service-admission procedure receives service, criticality, demand, and SLA classes and must accept, degrade, or reject the service.

The SDN/RIC contract assumes that reports encode finite classes and that acknowledgements are possible unless the environment has raised a modeled fault. It guarantees that each controller decision either reaches acknowledgement before D_{ctrl_ack} or produces a timeout/rollback outcome visible to the service layer. This prevents a local controller timeout from being hidden from SLA observers. The policy-control template is shown in Fig. 3.

Table 2. Controller procedures and bounded outcomes.

Procedure	Input channel	Main path	Permitted outcome
Rule miss	rule_miss?	Stable $\rightarrow$ miss $\rightarrow$ install pending or drop reason	flow_mod!, forward_cmd!, drop_report!, time-out report
Recovery	link_failure?, node_failure?	Stable $\rightarrow$ failure $\rightarrow$ standby switch or re-embedding $\rightarrow$ roll-back/stable	policy command, flow update, rollback, failure report
Sensing degradation	phy_kpi_report?, mac_report?	Normal $\rightarrow$ evaluate $\rightarrow$ sensing boost, constrained, or reject	policy command, service degraded, service rejected
Admission	service_request?	Idle $\rightarrow$ evaluate $\rightarrow$ normal, constrained, or reject	service accepted, degraded, or rejected

6 Application and Service Contracts

The application layer defines the service meaning of lower-layer events. It does not allocate radio or network resources directly; instead, it generates requests, declares SLA requirements, classifies service criticality, and receives accepted, degraded, rejected, or failed outcomes. This is necessary because the same PHY or MAC degradation may be tolerable for a best-effort telemetry flow but unacceptable for a safety-critical UAV-detection service.

The request automaton $\mathcal{A}_{REQ}$ creates service requests with finite classes for demand, criticality, mobility, and sensing need. The SLA automaton $\mathcal{A}_{SLA}$ tracks deadlines and service health. Observer automata monitor whether a request has been answered, whether a service has been degraded explicitly, and whether the degradation remains within the allowed budget. For a low-altitude UAV-detection scenario, the service contract may require high sensing freshness, bounded detection delay, limited false alarms, and priority recovery after link failure. For an industrial telemetry scenario, the same framework may prioritize bounded communication delay and packet delivery while allowing lower sensing resolution.

The service contract is expressed as

$$ServiceReq \Rightarrow \Diamond_{\leq D_{admission}} (Accepted \vee Degraded \vee Rejected), \quad (8)$$

and, for an active critical service,

$$Fault \Rightarrow \Diamond_{\leq D_{recovery}} (Recovered \vee Degraded \vee Failed). \quad (9)$$

These formulas define the property pattern; in UPPAAL they are implemented with observer clocks and finite outcome variables. The important modeling rule

Table 3. Representative channels and deadlines.

Channel	Direction	Deadline or observer bound
phy_kpi_report	PHY $\rightarrow$ MAC/SDN	report freshness bounded by D_{phy} and D_{mon}
mac_report	MAC $\rightarrow$ SDN/RIC	queue/resource report before controller decision
service_request	Application $\rightarrow$ SDN/RIC	admission answer within $D_{admission}$
sdn_policy_cmd	SDN/RIC $\rightarrow$ MAC/PHY	resource command acknowledged within D_{ctrl_ack}
rule_miss, flow_mod	Data plane $\leftrightarrow$ SDN/RIC	rule installed, forwarded, or dropped within D_{rule}
link_failure, node_failure	Environment/data plane $\rightarrow$ SDN/RIC	recovery, rollback, or failure report within $D_{recovery}$

The main clocks are c_{phy} for PHY reporting, c_{sched} for MAC scheduling, c_{mon} for telemetry age, c_{dec} for policy selection, c_{rule} for rule installation, c_{ack} for controller acknowledgement, c_{rec} for recovery, and c_{sla} for application deadlines. Each non-stable location has an invariant, for example $c_{dec} \leq D_{decision}$ in the controller evaluation location and $c_{sched} \leq D_{sched}$ in the scheduler location. Timeout transitions are explicit and lead to a visible report rather than to an unmodeled dead end.

Two consistency rules are enforced across the hierarchy. First, a lower-layer automaton may not silently discard a request that has an active service identifier. It must emit a drop, degradation, timeout, or failure report. Second, SDN/RIC must translate technical outcomes into service outcomes when the affected flow belongs to an admitted service. These rules connect resource verification to SLA verification.

8 Verification Properties

The model is intended for preliminary verification before detailed network simulation. The first group of properties concerns structural correctness:

$$A \Box \neg \text{deadlock}, \quad (10)$$

$$A \Box 0 \leq Q_i \leq Q_i^{max}, \quad (11)$$

$$A \Box Attempts_{rec} \leq Attempts_{rec}^{max}. \quad (12)$$

These formulas check deadlock freedom, bounded queues, and bounded recovery attempts. Reachability checks are then used for every meaningful location, including sensing boost, constrained allocation, rule drop, standby switch, rollback, accepted service, degraded service, and rejected service. An unreachable location indicates either an impossible scenario or an incomplete interface.

The second group concerns bounded response. For each request type k ,

$$request_k \Rightarrow \Diamond_{\leq D_k} outcome_k, \quad (13)$$

where $k \in \{phy, mac, rule, recovery, sensing, admission\}$. In UPPAAL this is encoded by observer automata because standard timed automata do not directly attach quantitative time bounds to all temporal operators. Each observer starts a clock at the request event and moves to an error location if no permitted outcome occurs before the corresponding deadline.

The third group checks cross-layer consistency:

$$A[] (RuleTimeout \wedge ActiveService) \Rightarrow ServiceImpact \neq NONE, \quad (14)$$

$$A[] (SensingLost \wedge CriticalService) \Rightarrow (Boosted \vee Degraded \vee Rejected), \quad (15)$$

$$A[] (FaultDetected \wedge ActiveService) \Rightarrow (Recovered \vee RolledBack \vee Failed \vee Degraded). \quad (16)$$

These properties prevent the controller from treating local technical recovery as sufficient when the service layer still observes SLA degradation.

The fourth group is comparative. The same service trace can be checked under three policy modes: a static communication/sensing split, a non-SDN local-control scheme, and the SDN/RIC-managed scheme. The comparison uses symbolic counters and observer states rather than stochastic frequencies. Metrics include accepted requests, degraded services, rejected services, rule misses, drops, recovery outcomes, queue overflows, and sensing-freshness violations. Frequency estimates, if required, should be added later through statistical model checking, priced/probabilistic timed automata, or finite traffic traces.

9 Discussion

The proposed hierarchy makes the boundary between physical phenomena and controller verification explicit. PHY equations and link-level simulation remain outside the timed automaton, but their results are abstracted into conservative finite classes. MAC scheduling is represented in enough detail to expose queues, resource conflicts, and drop reasons. SDN/RIC decisions are modeled as bounded control procedures with acknowledgements and rollback. The application layer turns lower-layer outcomes into SLA-visible service impact.

This division keeps the model compact enough for UPPAAL while preserving the causal chain from channel degradation to service-level violation. It also supports later refinement. The included screenshots identify the expected automata groups and therefore serve as a checklist for the model repository: PHY templates, MAC scheduling templates, SDN/RIC control templates, and application/SLA templates. When the XML templates are updated, the figures can be regenerated from the same repository.

The limitations are also explicit. The model does not estimate continuous SINR, CRB, or detection probability internally. It cannot by itself produce statistically meaningful rates of rule misses or packet drops without a stochastic extension or traffic trace. Finally, the discretization thresholds must be justified by separate radio and system-level analysis; otherwise, the formal model may be internally correct but poorly calibrated.

10 Conclusion

This paper consolidates separate PHY, MAC/resource scheduling, SDN/RIC, and application-layer specifications. The resulting model treats an SDN-managed 6G ISAC network as a hierarchy of synchronized timed automata with finite reports, explicit contracts, bounded deadlines, and service-impact propagation. It is designed for UPPAAL verification of deadlock freedom, bounded response, queue safety, rule-miss handling, sensing degradation, and failure recovery. The XML model repository can be used to regenerate the UPPAAL screenshots when the templates change and to run the listed observer queries on representative service traces.

Model repository. The UPPAAL XML models and corresponding screenshots are available in the public model repository:
<https://github.com/artmus208/uppaal-isac-model-repository>

Acknowledgments. The current manuscript assembles the formal layer descriptions into a single article and includes the available UPPAAL screenshots for the PHY, MAC, SDN/RIC, and application layers.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Ahmed, M., Nasir, A.A., Masood, M., Memon, K.A., Qureshi, K.K., Khan, F., Khan, W.U., Xu, F., Han, Z.: Advancements in UAV-based integrated sensing and communication: A comprehensive survey. arXiv preprint arXiv:2501.06526 (2025)
2. Alur, R., Dill, D.L.: A theory of timed automata. Theoretical computer science **126**(2), 183–235 (1994)
3. Aman, M., Rehman, G.U., Zubair, M., Alfakeeh, A., Aboulola, O.I., Daud, A.: AI-driven dynamic resource allocation for ISAC systems in 6G networks: intelligent beamforming, interference management, and power allocation. Scientific Reports (2026)
4. Bettira, R., Kahloul, L., Khalgui, M., Li, Z.: Reconfigurable hierarchical timed automata: Modeling and stochastic verification. In: 2019 IEEE international conference on systems, man and cybernetics (SMC). pp. 2364–2371. IEEE (2019)

5. Bogatyrev, V.A., Bogatyrev, S.V.: Combinatorial-probabilistic model of reliability of multi-level cluster systems taking into account dependent recovery of nodes at different levels. *Vestnik Tomskogo gosudarstvennogo universiteta. Upravlenie, vychislitel'naja tehnika i informatika – Tomsk State University Journal of Control and Computer Science* **74**, 51–61 (2026). <https://doi.org/10.17223/19988605/74/5>
6. Bogatyrev, V., Parshutina, S.: Redundant distribution of requests through the network by transferring them over multiple paths. In: *International Conference on Distributed Computer and Communication Networks*. pp. 199–207. Springer (2015)
7. Hossain, M.A., Xiang, A., Kiani, A., Saboorian, T., Kaippallimalil, J., Ansari, N.: AI-assisted E2E network slicing for integrated sensing and communication in 6G networks. *IEEE Internet of Things Journal* **11**(6), 10627–10634 (2023)
8. Liu, G., Han, L., Xi, R., Ma, L., Han, Z., Xue, Y., Zhao, H., Jin, J., Wang, Q., Xu, F.: Cooperative Sensing for 6G ISAC: Concept, Key Technologies, Performance Evaluation, and Field Trial. *Engineering* (2025)
9. Luo, H., Zhang, T., Zhao, C., Wang, Y., Lin, B., Jiang, Y., Luo, D., Gao, F.: Integrated sensing and communications framework for 6G networks. *IEEE Wireless Communications* **32**(6), 102–109 (2025)
10. Mustafin, A., Dukhanov, A.: Selective Standby Failover for Resilience-Aware Service Function Chaining. In: *2026 International Russian Smart Industry Conference (SmartIndustryCon)*. pp. 757–761. IEEE, Sochi, Russian Federation (Mar 2026). <https://doi.org/10.1109/SmartIndustryCon68821.2026.11492947>, <https://ieeexplore.ieee.org/document/11492947/>
11. Qaisar, M.U.F., Yuan, W., Günlü, O., Riihonen, T., Cui, Y., Zhang, L., Gonzalez-Prelcic, N., Di Renzo, M., Han, Z.: The role of isac in 6g networks: Enabling next-generation wireless systems. *IEEE Transactions on Network Science and Engineering* (2026)
12. Radosavljević, I., Bojović, P.D., Bojović, Ž.: RL-Based Resource Allocation in SDN-Enabled 6G Networks. *Future Internet* **17**(11), 497 (2025)
13. Tigane, S., Guerrouf, F., Hamani, N., Kahloul, L., Khalgui, M., Ali, M.A.: Dynamic timed automata for reconfigurable system modeling and verification. *Axioms* **12**(3), 230 (2023)